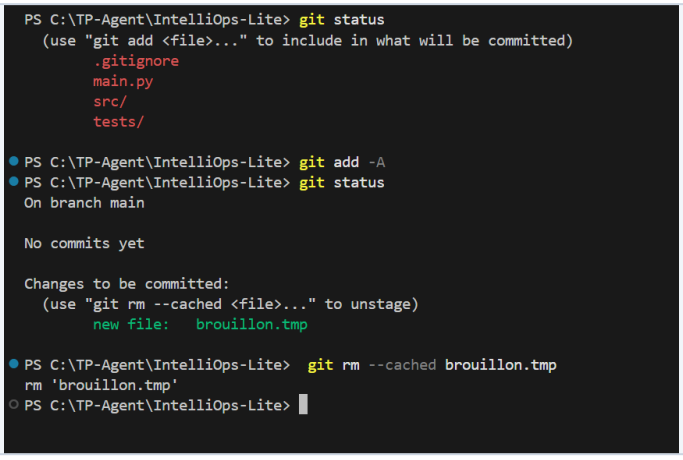
Rapport TP DevOps

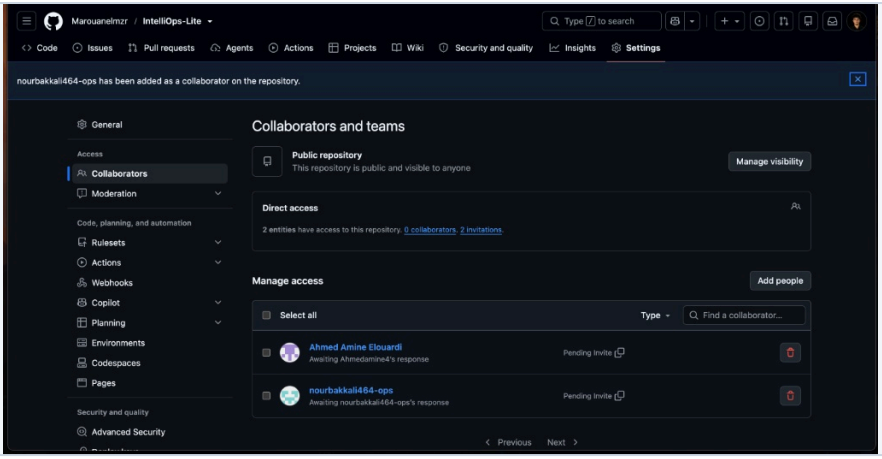
Git, GitHub et Intégration Continue (Projet IntelliOps-Lite)
Équipe : Marouane Elmozariahi, Nour Bakkali, Ahmed Amine Elouardi

Exercice 1 : Initialisation du projet

> **État d'un fichier :** Juste après `git add`, le fichier passe à l'état *indexé* (prêt à être commité). Juste après `git commit`, il passe à l'état *non modifié* par rapport à l'historique Git.
>
> **Action de `git rm --cached` :** Cette commande retire le fichier de l'index Git (le désindexe) sans le supprimer physiquement du disque local.
>
> **Rôle de `.gitignore` :** Il sert à exclure volontairement des fichiers ou des dossiers spécifiques du suivi de version Git, comme les dossiers `__pycache__` générés automatiquement par Python.



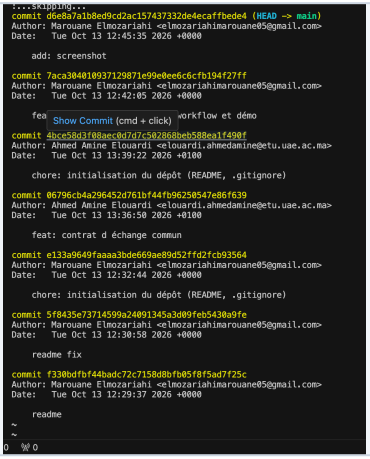
Retrait du fichier brouillon.tmp de l'index



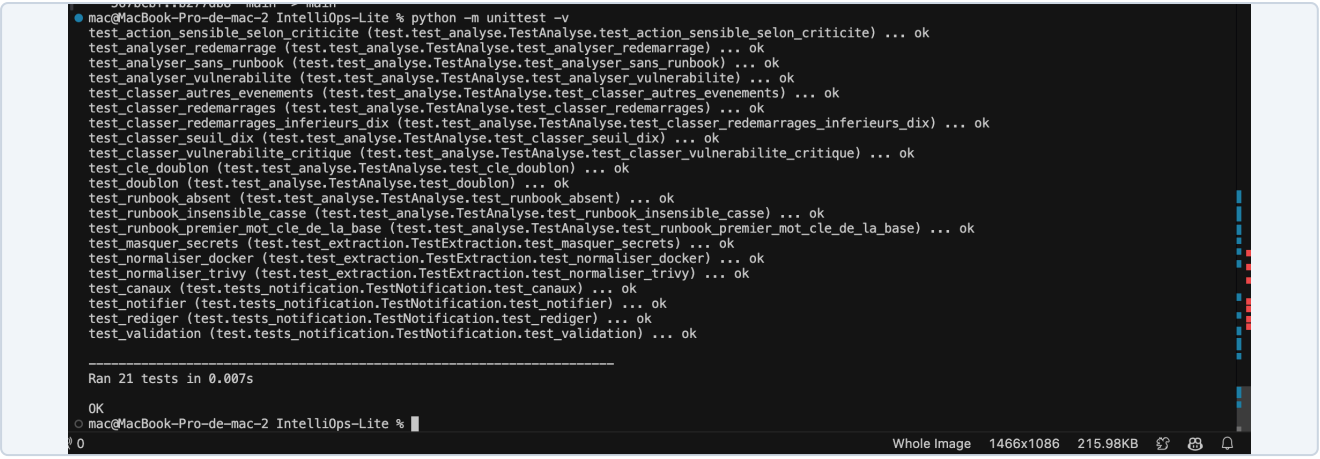
Ajout des collaborateurs sur le dépôt GitHub distant

Exercice 2 : Développement en parallèle

> **Rejet du push :** Le `git push` est rejeté car le dépôt distant contient des nouveaux commits (publiés par d'autres collaborateurs) qui ne sont pas encore présents dans l'historique du dépôt local.
>
> **Action de `git pull` :** Cette commande télécharge l'historique depuis le dépôt distant (fetch) et le fusionne avec la branche locale active (merge), la mettant ainsi à jour.
>
> **Dépôt local vs distant :** Le dépôt local est la copie de travail située sur la machine physique du développeur. Le dépôt distant (sur GitHub) est la version centralisée hébergée en ligne, servant de point de synchronisation pour l'équipe.



Historique des commits illustrant le travail en parallèle



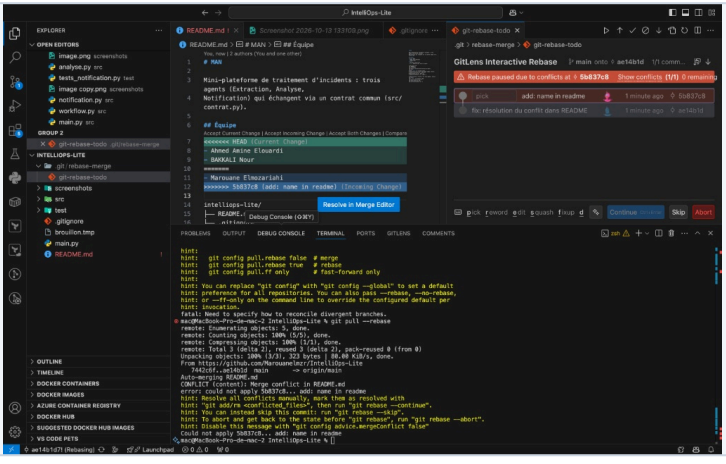
Exécution réussie des tests unitaires locaux

Exercice 3 : Intégration et conflit

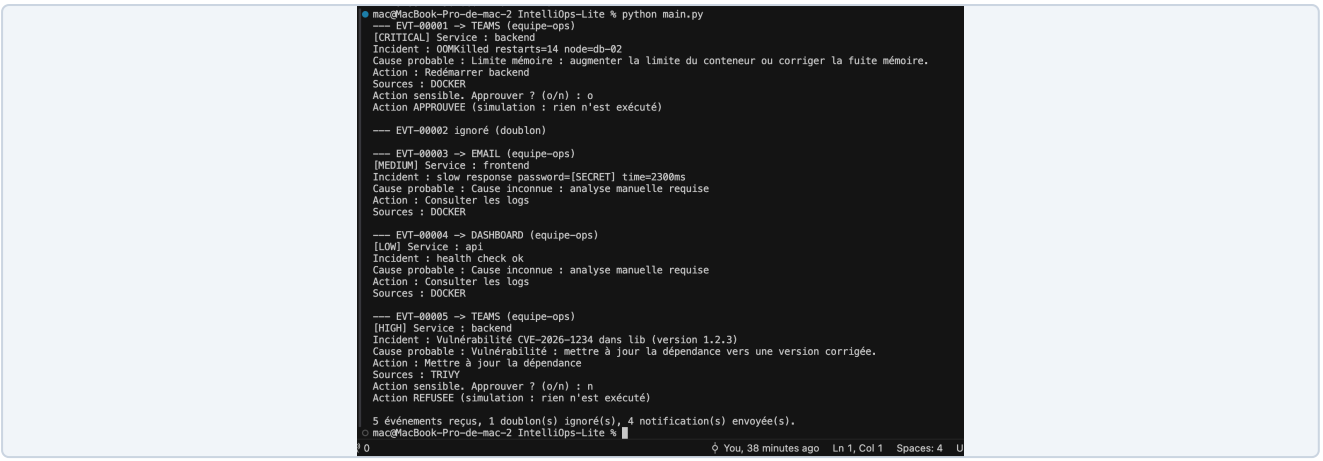
Apparition du conflit : Le conflit de fusion se déclenche parce que plusieurs développeurs ont modifié simultanément la même section (`## Équipe`) du même fichier (`README.md`) sur leurs dépôts locaux respectifs avant de pousser.

Résolution du conflit : La résolution manuelle consiste à ouvrir le fichier en conflit, repérer les balises de Git (<<<<<<<, =======, >>>>>>>), conserver les lignes pertinentes de chaque auteur, supprimer ces balises, puis valider avec `git add` et `git commit`.

L'avantage du contrat d'échange : En définissant strictement les structures de données partagées (Événement, Analyse, Notification), le contrat permet à chaque membre de développer sa partie en isolation totale. Lors de l'assemblage, l'intégration se fait sans heurts puisque les interfaces sont respectées.



Résolution du conflit de fusion dans l'éditeur (README.md)



Exécution finale de main.py avec les 3 agents intégrés